



Symbiosis Institute of Technology, Pune

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No.: 15

Name of Student	Akshit Gawade
PRN No.	25070122029
Batch	2025-2029
Class	A1
Academic Year & Semester	Second Year Semester III
Date of Submission	
Title of Assignment:	Write a Java program to demonstrate exception handling using finally, throw, and throws.
Problem Statement & Learning Objective	Problem Statement Write a Java program to demonstrate exception handling using finally, throw, and throws. Learning Objectives To implement finally, throw, and throws for effective exception management
Theory	The finally block always executes regardless of whether an exception occurs, making it suitable for resource cleanup. The throw keyword explicitly generates an exception, while throws declares exceptions that a method may pass to its caller. These mechanisms improve structured error handling and application stability.
Program Code	Main.java: <pre>class InvalidAgeException extends Exception { public InvalidAgeException(String m) { super(m); } }</pre>

```

public class Main {
    public static void validate(int age) throws
InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or
above");
        }
        System.out.println("Vlaid age : " + age);
    }

    public static void main (String[] args) {
        try {
            validate(12);
        } catch (InvalidAgeException e) {
            System.out.println("Caught Exception : " +
e.getMessage());
        }
    }
}

```

Main2.java:

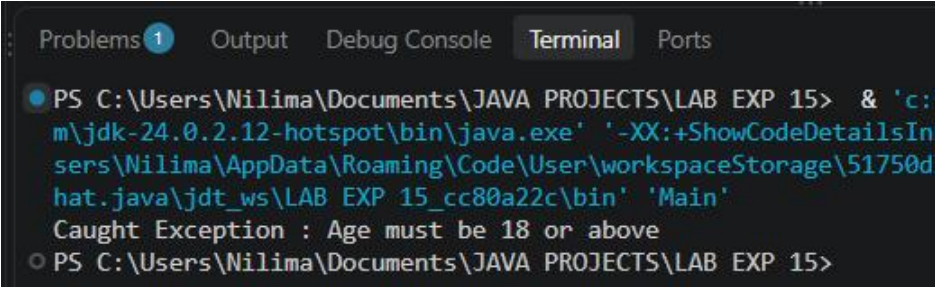
```

public class Main2 {
    public static void main (String[] args) {
        try {
            int[] n = new int[5];
            int divisor = 0;
            for (int i = 0; i < n.length; i++) {
                int res = n[i] / divisor;
                System.out.println(res);
            }
        } catch (ArithmeticException e) {
            throw new RuntimeException("Error : Division by zero
occured", e);
        }
    }
}

```

Output Screenshots

Age input: 12:

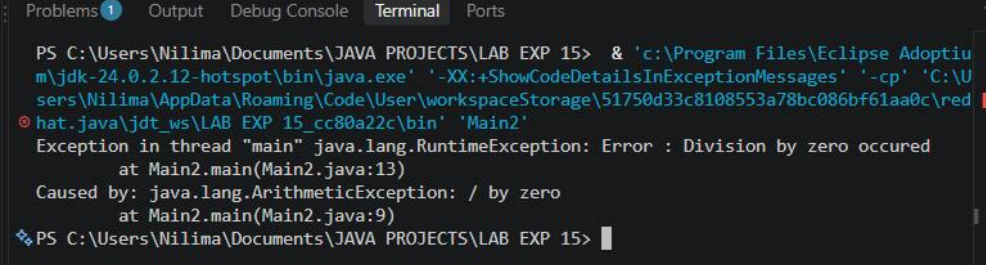


```

PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 15> & 'c:
m\jdk-24.0.2.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsIn
sers\Nilima\AppData\Roaming\Code\User\workspaceStorage\51750d
hat.java\jdt_ws\LAB EXP 15_cc80a22c\bin' 'Main'
Caught Exception : Age must be 18 or above
PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 15>

```

Divisor set to 0:

	
Exercise Qns and Code	1. Create a Login program that throws an exception for invalid password and uses finally block. <pre>import java.util.Scanner; public class Ex1 { public static void main (String[] args) { try (Scanner in = new Scanner(System.in)) { String correctPassword = "admin123"; try { System.out.print("Enter Password : "); String password = in.nextLine(); if (!password.equals(correctPassword)) { throw new Exception("Invalid Password!"); } System.out.println("Login Successful."); } catch (Exception e) { System.out.println("Error: " + e.getMessage()); } finally { System.out.println("Login process completed."); } } } }</pre> 2. Create an ATM PIN Verification program that throws an exception for an invalid PIN entered by the user and uses a finally block to display a message indicating that the verification process has completed. <pre>import java.util.Scanner; public class Ex2 { public static void main (String[] args) { try (Scanner in = new Scanner(System.in)) { try { String pass = "12345"; System.out.print("Enter ATM PIN : "); String pin = in.next(); if (!pin.equals(pass)) { throw new Exception("Incorrect ATM PIN!"); } } } } }</pre>

```

    }
    System.out.println("Logging in...");
    System.out.println("ATM Check in Verified!!!");
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("ATM Verification
Completed!");
}
}
}
}
}

```

Output Screenshots

Ex1:

```
Problems Output Debug Console Terminal Ports
```

- PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 15> & 'c:\Program Files\Eclipse Adoptiu
m\jdk-24.0.2.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\U
sers\Nilima\AppData\Roaming\Code\User\workspaceStorage\51750d33c8108553a78bc086bf61aa0c\red
hat.java\jdt_ws\LAB EXP 15_cc80a22c\bin' 'Ex1'
Enter Password : akshit
Error: Invalid Password!
Login process completed.
- PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 15> c.; cd 'c:\Users\Nilima\Documents\J
AVA PROJECTS\LAB EXP 15'; & 'c:\Program Files\Eclipse Adoptium\jdk-24.0.2.12-hotspot\bin\ja
va.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Nilima\AppData\Roaming\Co
de\User\workspaceStorage\51750d33c8108553a78bc086bf61aa0c\redhat.java\jdt_ws\LAB EXP 15_cc8
0a22c\bin' 'Ex1'
Enter Password : admin123
Login Successful.
Login process completed.
- PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 15> □

Ex2:

```

Problems    Output    Debug Console    Terminal    Ports
● PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 15> & 'c:\Program Files\Eclipse Adoptiu
m\jdk-24.0.2.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\U
sers\Nilima\AppData\Roaming\Code\User\workspaceStorage\51750d33c8108553a78bc086bf61aa0c\red
hat.java\jdt_ws\LAB EXP 15_cc80a22c\bin' 'Ex2'
Enter ATM PIN : 1000
Error: Incorrect ATM PIN!
ATM Verification Completed!
● PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 15> c::; cd 'c:\Users\Nilima\Documents\J
AVA PROJECTS\LAB EXP 15'; & 'c:\Program Files\Eclipse Adoptium\jdk-24.0.2.12-hotspot\bin\ja
va.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Nilima\AppData\Roaming\Co
de\User\workspaceStorage\51750d33c8108553a78bc086bf61aa0c\redhat.java\jdt_ws\LAB EXP 15_cc8
0a22c\bin' 'Ex2'
Enter ATM PIN : 12345
Logging in...
ATM Check in Verified!!!
ATM Verification Completed!
○ PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 15>

```

Conclusion

This experiment demonstrates controlled exception propagation and guaranteed resource cleanup. It highlights using `throw` to trigger exceptions, `throws` to declare potential caller hazards, and the `finally` block to execute critical cleanup operations regardless of program execution outcome.